SOP-IT-003 · Observer dashboards: local by default, remote access on purpose

SOP IT-003 | Status draft | Changed 2026-11-01 | For Stewards

Department: IT · **ID:** 003 · **Status:** draft (approved when tagged) · **Owner:** whoever runs an observer on their machine · **Last changed:** 2026-11-01

Purpose

A bot or observer that serves a dashboard from someone's computer must not answer the whole network by accident. This procedure says what "local by default" means, what changes for the people who used to open the page from another device, and how to give remote access back deliberately.

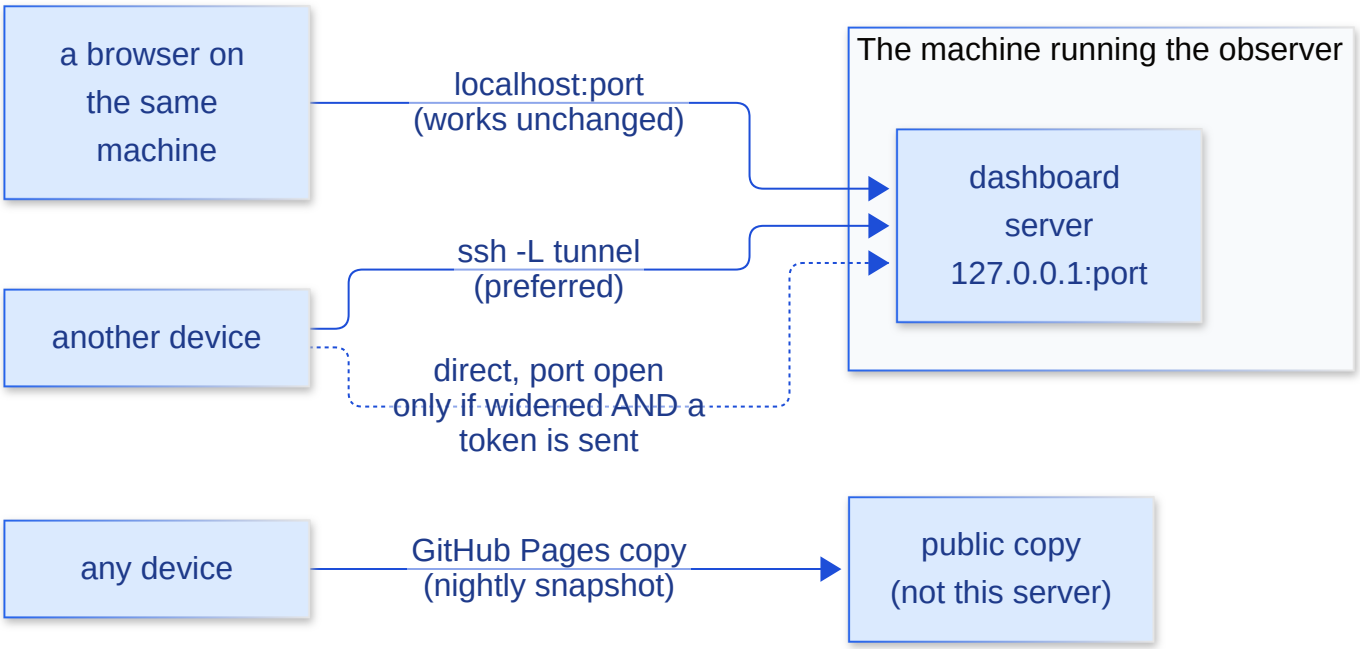
Scope

Every observer in the fleet that serves a page or an API from a person's machine. The worked example is a bot dashboard on port `<port>` that used to listen on every interface and now listens on loopback only.

What changes when a dashboard becomes local-only, and what you will notice

Before	After the monitor's next restart
The server listened on **all interfaces**, so a phone or laptop on the home or VPN network could open `http://<this-machine>:<port>`	It listens on **127.0.0.1** only. From any other device the page **times out and looks "down"**
Only POST routes checked who was calling; configuration, trades, logs, the journal and the alert history answered anyone	**Every** `/api` route and the live **WebSocket** refuse a caller that is not local (or not carrying the token, below)
`/api/config` showed the whole configuration	`/api/config` is **redacted**: keys, URLs and wallet fields no longer appear, even locally
—	`http://localhost:<port>` **on the machine itself works unchanged**; the **public GitHub Pages copy is unaffected** (it reads the nightly snapshot, not this server)

Three ways to reach the dashboard



Steps: give remote access back, on purpose

1. Decide that a second device really needs the live page. *You should see:* a reason you could write in one line. If it is just a glance, the public copy or a screenshot is enough.
2. **Preferred:** reach it through the Mac instead of widening the server: from the other device `ssh -L <port>:localhost:<port> <user>@<this-machine>`, then open `http://localhost:<port>` there. *You should see:* the dashboard, with nothing on the network changed.
3. **If the server itself must listen wider:** add both `BIND_HOST=0.0.0.0` **and** `BOT_API_TOKEN=<a long random value>` to the bot's `.env` (helpers never read `.env`; you edit it yourself), then restart the monitor (`sh scripts/edge-monitor-service.sh restart`). *You should see:* the log line `Server listening on 0.0.0.0:<port>`.
4. From the other device, send the token as a header: `Authorization: Bearer <value>` (a script, or a browser extension that sets headers). A `?token=` in the address bar also works but stays in the browser's history; use it only on your own device and clear it after. *You should see:* live data. Without the token the page shell loads but every API call and the socket are refused.
5. Never widen the binding without the token, never paste the token into a chat, an Issue or a commit, and remove both lines from `.env` when the need ends. *You should see:* the log line back to `127.0.0.1:<port>` after a restart.

Verification

From this machine, `curl -s localhost:<port>/api/edge/stats` answers. From another device without the token, the port times out (default) or `/api/edge/stats` returns 401 (widened). `curl -s localhost:<port>/api/config` shows no key, URL or wallet field.

Troubleshooting

Symptom	Likely cause	What to do
The dashboard "went down" on the phone after a restart	local-by-default, as designed	use step 2, or steps 3–4 on purpose
`401 Unauthorized` from another device	widened without the token, or the token is wrong	send the Bearer header; check `.env`

The page loads but shows no data from another device	the token is not reaching the API or socket	prefer the SSH tunnel (step 2); otherwise check the header

Where the heads-up came from: the "You should know" reader

The exposure was first spelled out by Claude Code's built-in **You should know** plugin (`cc-plugin-you-should-know@builtin`), a second reader that runs beside the main agent in a session. Its own heads-up on 2026-10-31 said what it is; keep these four points in mind whenever a helper works on an observer:

What it is	What it means for you
Turned on with `/plugin enable cc-plugin-you-should-know@builtin`. Reloading plugins is enough: **no `/restart` is needed**, so a restart refused because background work is running does not matter	to know it runs, look for its **Heads up** note in the session; that note is its own verification
A **second reader with no tools**: it cannot open files, check GitHub or confirm a deploy. It only re-reads the conversation and flags what may have been missed	a heads-up is a reason to check, never a check done. The main agent's requirements ledger, and this procedure's **Verification**, stay the real checklist
Each check is an extra model request on the same weekly allowance the session already uses	expect a small, steady cost while long sessions run; switch it off from `/plugin` when the allowance is tight
Its note offers three answers: **Understood**, **Chat in main session** (hands the point to the main agent), **Dismiss**	choose *Chat in main session* when the point needs a file opened or a command run: only the main agent can do that

Related

- SOP-IT-001 (VPN and DNS stalls), SOP-IT-002 (reading room and helper access)
- The observer's own runbook; the heads-up that first explained this change came from the session's "You should know" reader on 2026-10-31

Change log

Date	Change	By
2026-10-31	Drafted from the dashboard gate and the second reader's heads-up	Alfred
2026-10-31	The "You should know" reader recorded from its own heads-up (Christopher's screenshot): how to tell it runs, no tools, what it costs, its three answers	Alfred
2026-11-01	Lucid pass: Lucide icons, the three ways to reach a dashboard in Mermaid; the reader's next heads-up (base permission) is recorded in SOP-GOV-002	Alfred